Gate 05 Reconciliation: DRF API

Field	Value
Gate	05 — DRF API
Spec version	Original (pre-implementation)
Implementation date	2026-04-16
Reconciled by	Claude Code
Branch	gate/05-drf-api

Summary

Gate 05 was implemented as specced with all 12 endpoints, tier-aware serializers, GeoJSON map endpoints, dashboard aggregation, and OpenAPI schema. Security and code quality reviews identified improvements that strengthened the implementation beyond the original spec — most notably sensitive coordinate safety, defense-in-depth queryset scoping, query optimization on the dashboard, and a shared tier-detection mixin.

Acceptance Criteria Status

#	Criterion (from spec)	Status	Notes
1	All endpoints return correct HTTP status codes and response shapes	Implemented as specced	All 12 endpoints functional, 43 integration tests + 79 adversarial tests
2	Tier-gating integration tests pass	Implemented as specced	Field presence/absence verified for anonymous, Tier 2, Tier 3, and Tier 5
3	N+1 query tests pass for all list endpoints	Implemented as specced	`select_related`/`prefetch_related` on all viewsets; retrieve action prefetches additional relations
4	OpenAPI schema generates without errors	Implemented as specced	`drf-spectacular` at `/api/v1/schema/`, Swagger UI at `/api/v1/docs/`
5	`/api/v1/map/localities/` returns valid GeoJSON FeatureCollection	Implemented as specced	Uses `djangorestframework-gis` `GeoFeatureModelSerializer`
6	Sensitive record coordinates are redacted	Modified	Strengthened: serves null geometry when `location_generalized` is missing (instead of leaking exact coords)
7	All map filter parameters work correctly	Implemented as specced	All 8 filters: species_id, family, iucn_status, watershed_id, locality_type, presence_status, coordinate_precision, bbox

8	Map endpoints return HTTP 200 with empty collections	Implemented as specced	All three map endpoints return 200 with empty data structures
9	`djangorestframework-gis` is installed and configured	Implemented as specced	Already in requirements from Gate 02; `rest_framework_gis` in INSTALLED_APPS
10	Invoke test-writer for adversarial API tests	Implemented as specced	79 adversarial tests written covering tier escalation, malformed params, GeoJSON shape, sensitive coords
11	Invoke security-reviewer	Implemented as specced	All critical/high findings addressed
12	Invoke code-quality-reviewer	Implemented as specced	All critical/important findings addressed

User Story Status

Story	Title	Status	Notes
BE-05-1	Species List	Implemented as specced	Pagination with `described_count`/`undescribed_count`; all 5 filters + search + ordering
BE-05-2	Species Detail	Implemented as specced	Tier-aware assessments (accepted-only for public, all for Tier 3+); ex_situ_summary; field_programs
BE-05-3	Institution List/Detail	Implemented as specced	contact_email and species360_id hidden for Tier 1-2 via `to_representation`
BE-05-4	ExSituPopulation List/Detail	Modified	Queryset uses `.for_tier()` for defense-in-depth (spec only mentioned TierPermission)
BE-05-5	Field Program List/Detail	Implemented as specced	Nested focal_species and partner_institutions
BE-05-6	Dashboard	Modified	Uses single aggregation query instead of per-status loop; uses model constants
BE-05-7	Species Localities GeoJSON	Modified	Sensitive coordinate safety hardened; numeric filter params validated
BE-05-8	Watershed List	Implemented as specced	species_count via annotated subquery
BE-05-9	Map Summary	Implemented as specced	Cached with 5-minute TTL, signal-invalidated

Deviations

`django-filter` added as dependency

- **Spec said:** Filter parameters listed per endpoint but no filtering library specified

- **Implementation does:** Uses `django-filter` (`django_filters`) with DRF filter backend integration
- **Reason:** Standard DRF filtering library; provides declarative FilterSet classes with validation
- **Impact:** None — cleaner than manual query param parsing on viewsets

`drf-spectacular` added for OpenAPI (not `drf-yasg` or other)

- **Spec said:** "Generate OpenAPI schema with `drf-spectacular` "
- **Implementation does:** Exactly as specced — `drf-spectacular` with schema at `/api/v1/schema/` and Swagger UI at `/api/v1/docs/`
- **Impact:** None — matches spec

Species search uses `common_names__name` (not `commonname__name`)

- **Spec said:** `search` on `commonname__name`
- **Implementation does:** `search_fields = ["scientific_name", "provisional_name", "common_names__name"]`
- **Reason:** Django search follows the model's `related_name` , which is `common_names`
- **Impact:** None — correct Django ORM lookup syntax

Sensitive coordinate redaction hardened beyond spec

- **Spec said:** "The serializer serves `location_generalized` for records where `is_sensitive = True` "
- **Implementation does:** Serves `location_generalized` when available; serves **null geometry** when `is_sensitive=True` but `location_generalized` is NULL
- **Reason:** Security review C-1 identified that `queryset.update(is_sensitive=True)` bypasses `save()` , leaving `location_generalized` as NULL. Without this guard, exact coordinates would leak for supposedly-sensitive records.
- **Impact:** Stronger protection — no coordinate data served when generalization is missing

ExSituPopulation queryset uses `.for_tier()` defense-in-depth

- **Spec said:** `TierPermission(3)` gates endpoint access
- **Implementation does:** `TierPermission(3)` at permission layer AND `ExSituPopulation.objects.for_tier(tier)` at queryset layer
- **Reason:** Security review H-4 and code quality review CQ-4 identified that permission-only gating has no defense-in-depth. If the permission class is ever misconfigured, the queryset returns all records.
- **Impact:** Safer — data layer is self-defending regardless of permission configuration

Dashboard uses single aggregation instead of per-status COUNT loop

- **Spec said:** Response shape with `by_iucn_status` counts
- **Implementation does:** Single `values("iucn_status").annotate(c=Count("id"))` query instead of 9+ separate COUNT queries
- **Reason:** Code quality review CQ-2 identified the per-status loop as firing 9+ DB queries on a public, high-traffic endpoint
- **Impact:** Same response shape, fewer database round-trips

Conservation assessments use `.for_tier()` queryset method

- **Spec said:** "Serializers filter fields based on `request.user.access_tier` "
- **Implementation does:** `obj.conservation_assessments.for_tier(tier)` in serializer, aligning with the model's queryset API

- **Reason:** Security review M-1 — serializer-only filtering is fragile; the model already invested in a `.for_tier()` method
- **Impact:** Consistent with model API; defense-in-depth

Additions (not in spec)

TierAwareSerializerMixin

- `accounts/serializer_mixins.py` provides `_get_tier()` method shared by `SpeciesDetailSerializer` and `InstitutionDetailSerializer`. Eliminates duplicated tier-detection logic and ensures consistency across all tier-aware serializers.

Cache invalidation signals

- `species/signals.py` — invalidates dashboard cache on Species, ConservationAssessment, SpeciesLocality changes
- `populations/signals.py` — invalidates dashboard cache on ExSituPopulation and Institution changes
- Registered via `AppConfig.ready()` in both apps

Numeric filter param validation on map endpoint

- `species_id` and `watershed_id` query params are validated with `int()` in try/except to prevent 500 on non-numeric input. Security review H-2 identified this as an unhandled error path.

`django-filter` and `drf-spectacular` dependencies

- Added to `requirements.txt` and `INSTALLED_APPS`
- `DEFAULT_FILTER_BACKENDS` configured in `REST_FRAMEWORK` settings with `DjangoFilterBackend`, `SearchFilter`, `OrderingFilter`

Deferred Items

Item	Deferred to	Reason
Map localities result cap / pagination	Post-MVP	Security C-2; at MVP scale (~79 species) risk is low; standard for GeoJSON endpoints
Map localities caching	Post-MVP	Security M-2; acceptable at MVP scale with small dataset
Watershed list caching	Post-MVP	CQ-10; reference data that changes rarely
OpenAPI schema auth restriction	Post-MVP	Security L-3; schema is public documentation at MVP
ChoiceFilter validation on enum fields	Post-MVP	CQ-11; invalid values return empty results (safe, just not user-friendly)
Shared brief serializers across apps	Post-MVP	CQ-5; `_SpeciesBriefSerializer` / `_InstitutionBriefSerializer` duplicated but diverge intentionally
Rate limiting on X-Forwarded-For	Post-MVP	Security H-3; inherited from Gate 03; requires proxy configuration

Registration email enumeration	Post-MVP	Security L-1; inherited from Gate 03

Technical Decisions Made During Implementation

1. **`GeoFeatureModelSerializer` geometry swapping requires try/finally** — The serializer temporarily swaps `instance.location` with `instance.location_generalized` during `to_representation`. Without `try/finally`, an exception during serialization would leave the instance in a corrupted state. This is the safe mutation pattern for this library.

2. **`described_count` and `undescribed_count` are global, not filtered** — The species list pagination includes platform-wide counts regardless of active filters. This matches the spec's intent (directory header showing total inventory) but is documented here as a deliberate choice.

3. **`ExSituPopulationViewSet` does not institution-scope reads at API layer** — Unlike the Django Admin (Gate 04) which scopes the queryset to the user's institution, the API returns all populations for Tier 3+ users. This matches the spec: "all ExSituPopulation records for that species are returned (read is not institution-scoped; only writes are)."

4. **Dashboard cache key uses manual version** — `"api:dashboard:v1"` is hardcoded. Cache must be flushed on deploy if the response schema changes. Acceptable at MVP; production deployment should add automated cache flush.

Spec Updates Needed

- **Gate 05 spec** should note that `common_names__name` (not `commonname__name`) is the correct search field lookup
- **Gate 05 spec** should note that `django-filter` is required as a dependency
- **Gate 05 spec** should document that sensitive localities with missing `location_generalized` serve null geometry (not exact coordinates)
- **Architecture proposal** should document the decision that API reads are not institution-scoped (unlike admin writes)